

מבוא למדעי המחשב

הרצאה 2 : משתנים, אופרטורים

המרה נתונים

```
/* program converts the distance of a marathon  
26 miles and 385 yards to kilometers */
```

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int miles, yards;
```

```
    double kilometers;
```

```
    miles = 26;
```

```
    yards = 385;
```

```
    kilometers = 1.609*(miles+yards/1760.0);
```

```
    printf ("A marathon is %lf kilometers\n", kilometers);
```

```
}
```

המרה נתונים

```
/* program converts the distance of a marathon  
26 miles and 385 yards to kilometers */
```

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int miles, yards;
```

```
    double kilometers;
```

← הצהרת משתנים

```
    miles = 26;
```

```
    yards = 385;
```

```
    kilometers = 1.609*(miles+yards/1760.0);
```

```
    printf ("A marathon is %lf kilometers\n", kilometers);
```

```
}
```

הצהרת משתנים

- הקצאת **מקומות בזיכרון**, שהתכנית תוכל לשמור בהם ערכים, לשנות אותם ולקרוא מהם את הערכים שאוחסנו בהם.
- לכל מקום כזה נותנים **שם**, כדי שהתכנית תוכל להתייחס אליו.
- שם של משתנה מורכב מאותיות, ספרות וקו תחתני. התו הראשון לא יכול להיות ספרה.
- יש הבדל בין אותיות גדולות וקטנות : $A \neq a$
- ההצהרה מגדירה טיפוס (סוג) של משתנה.
- למשל, miles מטיפוס **int** (integer, מספר שלם).
- גודל הזיכרון לצורך שמירת הערך של המשתנה תלוי בסוג שלו.

תמונת הזכרון

/* program converts the distance of a marathon
26 miles and 385 yards to kilometers */

#include <stdio.h>

void main()

{

int miles, yards;

double kilometers;

miles = 26;

yards = 385;

kilometers = 1.609*(miles+yards/1760.0);

printf ("A marathon is %lf kilometers\n", kilometers);

}

miles

yards

kilometers

?

?

?

המרה נתונים

```
/* program converts the distance of a marathon  
26 miles and 385 yards to kilometers */
```

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int miles, yards;
```

```
    double kilometers;
```

```
    miles = 26;
```

```
    yards = 385;
```

← השמה

```
    kilometers = 1.609*(miles+yards/1760.0);
```

```
    printf ("A marathon is %lf kilometers\n", kilometers);
```

```
}
```

תמונת הזכרון

/* program converts the distance of a marathon
26 miles and 385 yards to kilometers */

#include <stdio.h>

void main()

{

int miles, yards;

double kilometers;

miles = 26;

yards = 385;

kilometers = 1.609*(miles+yards/1760.0);

printf ("A marathon is %lf kilometers\n", kilometers);

}

miles

yards

kilometers

26

?

?

תמונת הזכרון

/* program converts the distance of a marathon
26 miles and 385 yards to kilometers */

#include <stdio.h>

void main()

{

int miles, yards;

double kilometers;

miles = 26;

yards = 385;

kilometers = 1.609*(miles+yards/1760.0);

printf ("A marathon is %lf kilometers\n", kilometers);

}

miles

yards

kilometers

26

385

?

תמונת הזכרון

/* program converts the distance of a marathon
26 miles and 385 yards to kilometers */

#include <stdio.h>

void main()

{

int miles, yards;

double kilometers;

miles = 26;

yards = 385;

kilometers = 1.609*(miles+yards/1760.0);

printf ("A marathon is %lf kilometers\n", kilometers);

}

miles

yards

kilometers

26

385

42.185969

פעולות חשבון בסיסיות

■ **+** ו **-** : חיבור וחסור ערכים

■ ***** ו **/** : כפל וחלוקת ערכים.

אם בחלוקה משתתפים שני ערכים שלמים אז גם תוצאה – ערך שלם!!

למשל: $6/3 \rightarrow 2$ $7/3 \rightarrow 2$ $8/3 \rightarrow 2$
 $6\%3 \rightarrow 0$ $7\%3 \rightarrow 1$ $8\%3 \rightarrow 2$

■ **%** : שארית החלוקה (עבור מספרים שלמים!)
למשל: $6\%3 \rightarrow 0$ $7\%3 \rightarrow 1$ $8\%3 \rightarrow 2$

פעולות חשבון בסיסיות

■ **+** ו **-** : חיבור וחסור ערכים

■ ***** ו **/** : כפל וחלוקת ערכים.

אם בחלוקה משתתפים שני ערכים שלמים אז גם תוצאה – ערך שלם!!

■ **%** : שארית החלוקה (עבור מספרים שלמים!)

למשל: $6\%3 \rightarrow 0$ $7\%3 \rightarrow 1$ $8\%3 \rightarrow 2$

שימו לב: הביטוי $n \% m$ שומר את ה**סימן** של m : 

$(-8)\%3 \rightarrow -2$ $(-7)\%3 \rightarrow -1$

$(-8)\%(-3) \rightarrow -2$

פעולות חשבון בסיסיות

"פעולות" נקראות גם "אופרטורים"

- **סדר קדימות** של חישוב אופרטורים אריתמטיים (כלומר, באיזה סדר מבצעים את הפעולות בביטויים מורכבים), הוא כפי שמוגדר במתמטיקה. כלומר:
 - סוגריים $()$
 - כפל $*$, חילוק $/$, מודולו $\%$
 - חיבור $+$ חיסור $-$
 - השמה $=$

פלט

```
printf("A marathon is %lf kilometers\n", kilometers);
```



- ניתן לכלול בהדפסה של printf תוכן של משתנים.
- לצורך כך צריך לציין מהו סוג המשתנה באמצעות סימון מיוחד שלא מודפס.
- `%lf` אומר שהמשתנה מכיל מספר ממשי.
- לאחר המחרוזת (בין " ל-") צריך לציין מאיזה משתנה לקחת את הערך להדפסה.

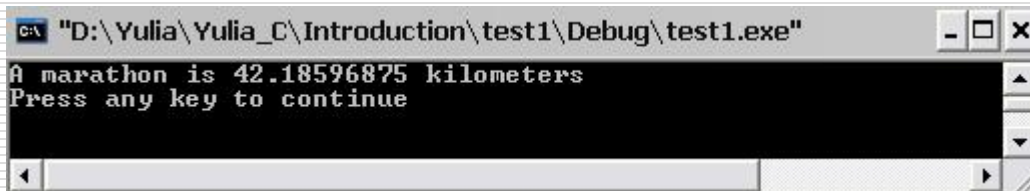
פלט

`printf("A marathon is %lf kilometers\n", kilometers);`



```
C:\ "D:\Yulia\Yulia_C\Introduction\test1\Debug\test1.exe"
A marathon is 42.185969 kilometers
Press any key to continue
```

`printf("A marathon is %.8lf kilometers\n", kilometers);`



```
C:\ "D:\Yulia\Yulia_C\Introduction\test1\Debug\test1.exe"
A marathon is 42.18596875 kilometers
Press any key to continue
```

ניהול עיצוב הדפסה:

`% cnt1.cnt2 type` – cnt1 - מספר תווים בהדפסה
cnt2 - מספר תווים אחרי נקודה

Format specifiers

%d	Integer (int)
%c	Character (char)
%f	Floating point (float)
%lf	Floating point (double)

תמונת הזכרון

/* program converts the distance of a marathon
26 miles and 385 yards to kilometers */

#include <stdio.h>

void main()

{

int miles, yards;

double kilometers;

miles = 26;

yards = 385;

kilometers = 1.609*(miles+yards/1760.0);

printf ("A marathon is %lf kilometers\n", kilometers);

}

miles

yards

kilometers

26

385

42.185969

תמונת הזכרון האמיתי

/* program converts the distance of a marathon
26 miles and 385 yards to kilometers */

#include <stdio.h>

void main()

{

int miles, yards;

double kilometers;

miles = 26;

yards = 385;

kilometers = 1.609*(miles+yards/1760.0);

printf ("A marathon is %lf kilometers\n", kilometers);

}

miles

yards

kilometers

11010

110..01

.....

ספירה בבסיס 2 (בינארי) Binary system

Decimal value	Binary value
0	0
1	1
2	10
3	11
4	100
.....	
9	1001
10	1010

שיטות ספירה מקומיות

- מספר מיוצג כרצף של ספרות
- לכל מיקום במספר משייכים "משקל" (ספרת האחדות, עשרות, מאות וכו').
- למשל, בשיטה העשרונית, המשקל של כל ספרה הוא 10 בחזקת המיקום שלה.
- הערך שמיוצג ע"י מספר הינו סכום הספרות מוכפלות במשקלן.

$$4903 = 4 \times 10^3 + 9 \times 10^2 + 0 \times 10^1 + 3 \times 10^0$$

ספירה בבסיס 2 (בינארי)

9	3	7	5
*	*	*	*
10^3	10^2	10^1	10^0

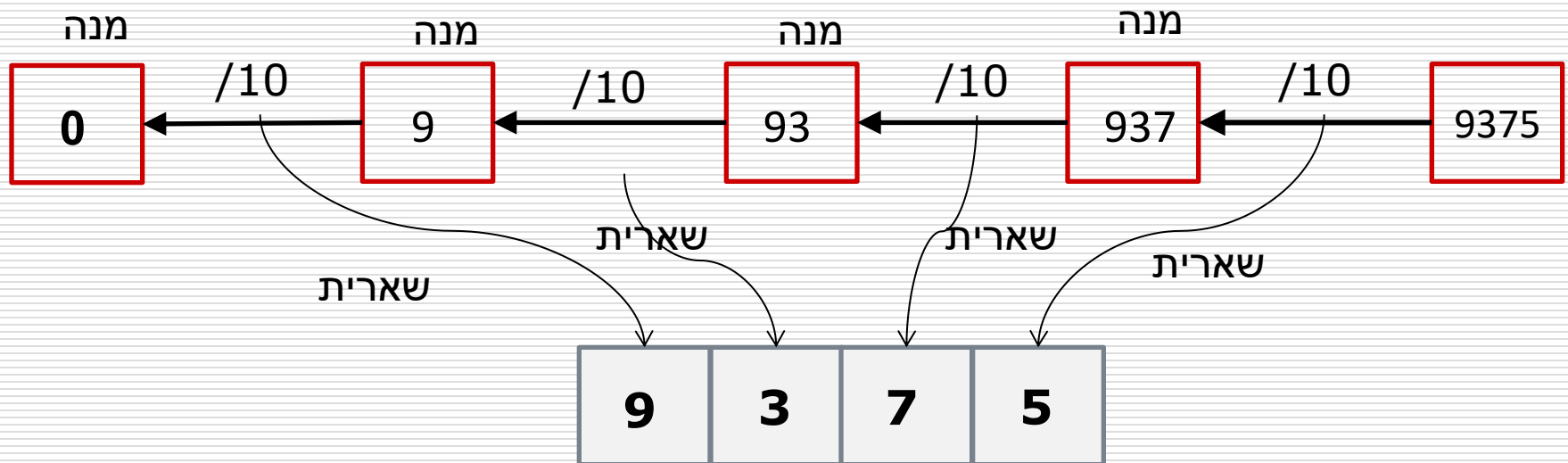
$$\begin{aligned} & 5 * 10^0 + 7 * 10^1 + 3 * 10^2 + 9 * 10^3 \\ &= 5 + 70 + 300 + 9000 \\ &= 9375 \end{aligned}$$

1	1	0	1
*	*	*	*
2^3	2^2	2^1	2^0

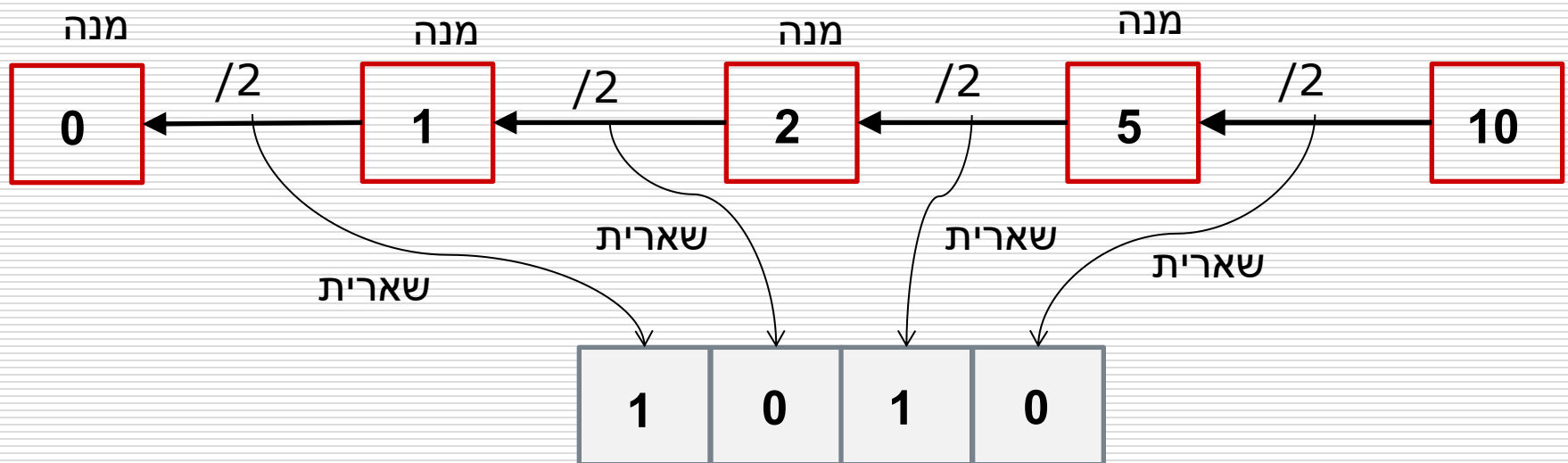
$$\begin{aligned} & 1 * 2^0 + 0 * 2^1 + 1 * 2^2 + 1 * 2^3 \\ &= 1 + 0 + 4 + 8 \\ &= 13 \end{aligned}$$

המרה מספר בינארי לעשרוני

ייצוג של מספר בבסיס 10 (עשרוני)



ייצוג של מספר בבסיס שתיים (בינארי)



המרה מספר עשרוני לבינארי

1	11	21	31	41	51
3	13	23	33	43	53
5	15	25	35	45	55
7	17	27	37	47	57
9	19	29	39	49	59

2	11	22	31	42	51
3	14	23	34	43	54
6	15	26	35	46	55
7	18	27	38	47	58
10	19	30	39	50	59

4	13	22	31	44	53
5	14	23	36	45	54
6	15	28	37	46	55
7	20	29	38	47	60
12	21	30	39	52	

8	13	26	31	44	57
9	14	27	40	45	58
10	15	28	41	46	59
11	24	29	42	47	60
12	25	30	43	56	

16	21	26	31	52	57
17	22	27	48	53	58
18	23	28	49	54	59
19	24	29	50	55	60
20	25	30	51	56	

32	37	42	47	52	57
33	38	43	48	53	58
34	39	44	49	54	59
35	40	45	50	55	60
36	41	46	51	56	

**There are 10 types of people
in this world.**

**Those who understand binary
and those who don't.**

ייצוג מספרים בבסיס כלשהו

- ניתן לייצג מספרים לפי הערך המקומי גם במערכות מספרים שונות

- במערכת הדצימאלית (בסיס 10) משתמשים בעשר ספרות לייצוג המספרים: $0 - 9$.

- במערכת מספרים בינארית יש רק שתי ספרות: $0, 1$.

- בכל בסיס, רצף חוקי של ספרות מקבל **משמעות נומרית** שונה, כלומר **ערך שונה**, כאשר כל ספרה מקבלת משקל לפי מיקומה היחסי.

- כשרוצים לציין שיש להתייחס לרצף ספרות כאל מספר בבסיס מסויים, כותבים את הבסיס בכתב תחת. למשל:
 10_2 הוא מספר בינארי ואילו 10_{10} הוא מספר דצימאלי.

ייצוג מספרים בבסיס כלשהו - המשך

■ בהינתן מספר בבסיס r :

$$d_{n-1} \cdots d_0$$

■ הערך המספרי שהוא מייצג הוא:

$$d_0 \times r^0 + d_1 \times r^1 + \cdots + d_{n-1} \times r^{n-1}$$

בסיסי מיוחד: 16 (Hexadecimal)

■ הספרות ההקסדצימליות:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.



■ ערך של מספר הקסדצימלי:

C=12

$$\begin{aligned} 7 * 16^0 + C * 16^1 + 0 * 16^2 + 2 * 16^3 &= \\ 7 + 192 + 0 + 8192 &= \\ = 8391 \end{aligned}$$

2	0	C	7
*	*	*	*
16^3	16^2	16^1	16^0

Decimal value	Hex value	Binary value
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Decimal value	Hex value	Binary value
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

$10_2 = ?$

$10_{16} = ?$

Decimal value	Hex value	Binary value
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

$$10_2 = 2_{10}$$

$$10_{16} = 16_{10}$$

ASCII code

Symbol תו	Binary value בינארי	Hex value הקסדצימלי	Decimal value עשרוני
A	0100 0001	41	65
a	0110 0001	61	97
B	0100 0010	42	66
M	0100 1101	4D	77
1	0011 0001	31	49
5	0011 0101	35	53
	0010 0000	20	32
?	0011 1111	3F	63

ASCII code

Symbol תו	Binary value בינארי	Hex value הקסדצימלי	Decimal value עשרוני
A	0100 0001	41	65
a	0110 0001	61	97
B	0100 0010	42	66
M	0100 1101	4D	77
1	0011 0001	31	49
5	0011 0101	35	53
Space	0010 0000	20	32
?	0011 1111	3F	63

- There are four *standard signed integer types*: **char**, **short**, **int**, **long**.

■ דוגמה להצהרה ראשונית:

- **int** miles, yards;
- **long** distance;
- **char** letter;

- There are four *standard signed integer types*: **char**, **short**, **int**, **long**.

■ דוגמה להצהרה ראשונית:

- **int** miles, yards;
- **long** distance;
- **char** letter;

■ יחד עם הצהרה אפשר גם לאתחל ערכים ראשוניים:

- **int** miles=26, yards;

- There are three *real floating types*:
float, double, long double

■ דוגמה להצהרה ראשונית:

- **double** x, y=3.6, min=-45.12345;

- int – %d (בבסיס 10 כרגיל)
- int – %x (בבסיס 16)
- char – %c
- float – %f (הצגה עם נקודה עשרונית 21.54)
- double – %lf (הצגה עם נקודה עשרונית 21.54)
- %e (הצגה עם מעריך 2.154e+001)

ASCII code

characters	'a'	'b'	'c'		'z'
values	97	98	99		112
characters	'A'	'B'	'C'		'Z'
values	65	66	67		90
characters	'0'	'1'	'2'	...	'9'
values	48	49	50		57
characters	'&'	'*'	'+'		
values	38	42	43		

התאמה בין int ל-char

```
int x=49;  // the same: x='1'  
char y='a'; // the same: y=97  
printf ("x=%d  y=%c", x, y);
```

התאמה בין int ל-char

int x=49; // the same: x='1'

char y='a'; // the same: y=97

printf ("x=%d y=%c", x, y);

פלט: x=49 y=a

התאמה בין int ל-char

int x=49; // the same: x='1'

char y='a'; // the same: y=97

printf ("x=%d y=%c", x, y);

x=49 y=a פלט:

printf ("x=%c y=%d", x, y);

x=1 y=97 פלט:

התאמה בין int ל-char

int x=49; // the same: x='1'

char y='a'; // the same: y=97

printf ("x=%d y=%c", x, y);

פלט: x=49 y=a

printf ("x=%c y=%d", x, y);

פלט: x=1 y=97

printf ("x_new=%d y_new=%d", x+1, y+1);

התאמה בין int ל-char

int x=49; // the same: x='1'

char y='a'; // the same: y=97

printf ("x=%d y=%c", x, y);

פלט: x=49 y=a

printf ("x=%c y=%d", x, y);

פלט: x=1 y=97

printf ("x_new=%d y_new=%d", x+1, y+1);

פלט: x_new=50 y_new=98

התאמה בין int ל-char

int x=49; // the same: x='1'

char y='a'; // the same: y=97

printf ("x=%d y=%c", x, y);

פלט: x=49 y=a

printf ("x=%c y=%d", x, y);

פלט: x=1 y=97

printf ("x_new=%d y_new=%d", x+1, y+1);

פלט: x_new=50 y_new=98

printf ("x_new=%c y_new=%c", x+1, y+1);

התאמה בין int ל-char

int x=49; // the same: x='1'

char y='a'; // the same: y=97

printf ("x=%d y=%c", x, y);

פלט: x=49 y=a

printf ("x=%c y=%d", x, y);

פלט: x=1 y=97

printf ("x_new=%d y_new=%d", x+1, y+1);

פלט: x_new=50 y_new=98

printf ("x_new=%c y_new=%c", x+1, y+1);

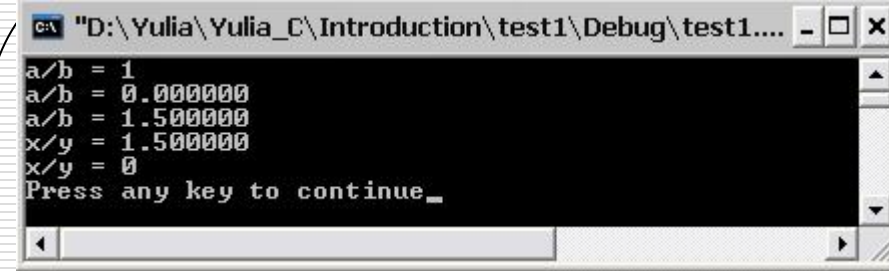
פלט: x_new=2 y_new=b

```
int a=3,b=2;  
double x=3, y=2;  
printf("a/b = %d\n",a/b);  
printf("a/b = %lf\n",a/b);  
printf("a/b = %lf\n",(double)a/b);  
printf("x/y = %lf\n",x/y);  
printf("x/y = %d\n",x/y);
```

אם רוצים שהמחשב יתייחס למשתנה
באופן חד-פעמי כבעל טיפוס אחר, אז
ניתן לציין זאת במפורש. זה נקרא
.casting

```
int a=3,b=2;  
double x=3, y=2;  
printf("a/b = %d\n",a/b);  
printf("a/b = %lf\n",a/b);  
printf("a/b = %lf\n",(double)a/b);  
printf("x/y = %lf\n",x/y);  
printf("x/y = %d\n",x/y);
```

אם רוצים שהמחשב יתייחס למשתנה
באופן חד-פעמי כבעל טיפוס אחר, אז
ניתן לציין זאת במפורש. זה נקרא
.casting



```
Microsoft Windows [Version 6.0.6002.18005]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\>"D:\Yulia\Yulia_C\Introduction\test1\Debug\test1..."
a/b = 1
a/b = 0.000000
a/b = 1.500000
x/y = 1.500000
x/y = 0
Press any key to continue_
```

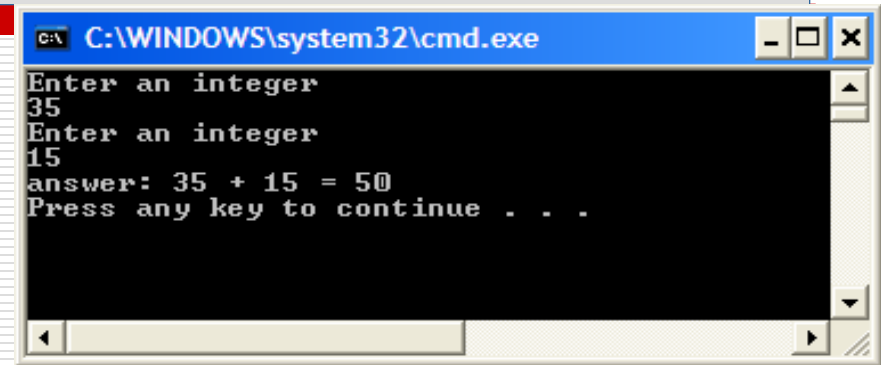
- כתבו תכנית הקולטת שני מספרים שלמים, מחשבת ומדפיסה סכומם.

- הצעה לפתרון:

- קלוט ערכים שלמים למשתנים a ו- b
- הצב סכומם במשתנה sum
- הדפס sum

```
#include <stdio.h>
void main()
{
    int first, second, sum;
    printf("Enter an integer\n");
    scanf("%d", &first);
    printf("Enter an integer\n");
    scanf("%d", &second);
    sum = first + second;
    printf("answer: %d + %d = %d\n", first, second, sum);
}
```

```
#include <stdio.h>
void main()
{
    int first, second, sum;
    printf("Enter an integer\n");
    scanf("%d", &first);
    printf("Enter an integer\n");
    scanf("%d", &second);
    sum = first + second;
    printf("answer: %d + %d = %d\n", first, second, sum);
}
```



```
C:\WINDOWS\system32\cmd.exe
Enter an integer
35
Enter an integer
15
answer: 35 + 15 = 50
Press any key to continue . . .
```

- ברוב בעיות אנו לא יודעים מראש ערכים שצריך לעבד אותם.
- לצורך זה יש לקלוט ערכים כאלה מהמשתמש ולשמור במשתנים שבחרנו.
- פונקציה ראשונה שמאפשרת לקלוט: `scanf()`

פונקציה scanf()

■ דוגמה:

- `scanf("%d", &a);`
 - `%d` – קוד מיוחד, מציין ל-C לקלוט מספר שלם (כמו בפלט)
 - סימן `&` מציין כתובת בזיכרון המחשב שבה נמצא המשתנה שישמור הערך (בדוגמה- משתנה `a`)

ייצוג כללי לשימוש `scanf`:

`scanf ("ייצוג%",&משתנה);`

```
#include <stdio.h>
void main()
{
    int first, second, sum;
    printf("Enter two integers\n");
    scanf("%d%d", &first, &second);
    sum = first + second;
    printf("answer: %d + %d = %d\n", first, second, sum);
}
```

- קוד למכונת צילום מוגדר כסכום 2 ספרות אחרונות ועוד 2 ספרות ראשונות של ת.ז.
כתבו תכנית בשפת C אשר מקבלת ערך ת.ז.
ומדפיסה קוד אישי לשימוש במכונת צילום.

- דוגמה: עבור ת.ז. 123456789
החישוב: $89 + 12$
והפלט: 101

- קלוט למשתנה id ערך ת"ז
- שארית המנה id ב- 100 הצב למשתנה end
- מנה של id ב- 100000000 הצב ב- start
- חבר end עם start והצב ב- code
- הדפס code

```
#include <stdio.h>

void main()
{
    long id;
    int start, end, code;
    printf("enter id\n");
    scanf("%ld",&id);
    start = id / 100000000;
    end = id % 100;
    code = start + end;

    printf("Code : %d \n", code);
}
```

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    long id;
```

```
    int start, end, code;
```

```
    printf("enter id\n");
```

```
    scanf("%ld",&id);
```

```
    start = id / 100000000;
```

```
    end = id % 100;
```

```
    code = start + end;
```

```
    printf("Code : %d \n", code);
```

```
}
```

הצהרה

קלט

חישובים

הדפסה תוצאות

משפט הצבה מקוצר

- לביטוי כדוגמת: $x = x + 2 ;$
ניתן לכתוב ביטוי שקול: $x += 2 ;$
הפרוש הוא ש- x מקבל את ערכו הקודם $+ 2$.
- באופן כללי עבור האופרטורים:
 $+, -, *, / , %$ ניתן לבצע את הקיצור
לדוגמה:
הביטוי $x *= y + 1 ;$
שקול לביטוי $x = x * (y + 1) ;$
ולא לביטוי $x = x * y + 1 ;$

הגדלה והקטנה עצמית

■ ++ אופרטור זה מוסיף 1 (increment).

■ -- אופרטור זה מוריד 1 (decrement).

$x++;$ $\Leftrightarrow x=x+1;$ $x--;$ $\Leftrightarrow x=x-1;$

■ אפשר להשתמש למשתנה מכל טיפוס

■ אפשר לשלב בביטוי מורכב :

$a=b++ - c;$

■ אופרטורים אלו יכולים להופיע בשתי צורות:

Pre increment/ decrement $++i;$ $--j;$

Post increment/ decrement $k++;$ $a--;$

הגדלה והקטנה עצמית

■ אם האופרטור נמצא מימין למשתנה (הגדלה/הקטנה מאוחרת), לדוגמה $++n$, אז לחישוב הביטוי משתמשים בערך של n ולאחר מכן מגדילים n ב-1.

■ אם האופרטור נימצא משמאל למשתנה (קדם הגדלה/הקטנה), לדוגמה $n++$, אז קודם מגדילים n ב-1, ולאחר מכן משתמשים בערך של n חדש לחישוב הביטוי.

■ כאשר $++n$ או $++n$ מופיעים כמשפט עצמאי (לא כחלק מביטוי או משפט השמה) אין הבדל בין שמאל לימין

מה זה "אחר מכן"?

```
void main()
{
    int x=5, y=8, z;

    z = x++ + --y;
    printf (" x=%d, y=%d, z=%d \n", x,y,z);
    z = x-- + y++;
    printf (" x=%d, y=%d, z=%d \n", x,y,z);
}
```

מה יודפס אחר ביצוע התכנית?

מה זה "אחר מכן"?

```
void main()  
{  
    int x=5, y=8, z;  
  
    z = x++ + --y;  
    printf (" x=%d, y=%d, z=%d \n", x,y,z);  
    z = x-- + y++;  
    printf (" x=%d, y=%d, z=%d \n", x,y,z);  
}
```



```
C:\ "D:\Yulia\Yulia_C\Introduction\t...  
x=6, y=7, z=12  
x=5, y=8, z=13  
Press any key to continue_
```

מה יודפס אחר ביצוע התכנית?